National Centre for Computing Education

Raspberry Pi

Year 7 — Programming I

Unit introduction

This unit is the first programming unit of KS3. The aim of this unit and the following unit ('programming 2') is to build learners' confidence and knowledge of the key programming constructs. Importantly, this unit does not assume any previous programming experience, but it does offer learners the opportunity to expand on their knowledge throughout the unit.

The main programming concepts covered in this unit are sequencing, variables, selection, and count-controlled iteration. All of the examples and activities for this unit use Scratch 3.

Overview of lessons

Lesson	Brief overview	Learning objectives
Lesson 1: Introduction to programming and sequencing	Learners will be introduced to the unit and will take part in an activity to help them understand the precise nature of instructions that computers need to execute. Learners will be taught the song *Frère Jacques* before working in pairs to place blocks of code into the appropriate subroutines so that their program will play the song correctly.	• Compare how humans and computers understand instructions (understand and carry out) • Define a sequence as instructions performed in order, with each executed in turn

		• Predict the outcome of a simple sequence • Modify a sequence
Lesson 2: Sequence and variables	In this lesson the learners will be introduced to variables as well as the opportunity to get more confident with sequences. The lesson will start with a story that includes variables; learners have to replace the variable names with the values they refer to when they reach the relevant places in the story. Learners will then be given a Scratch program where they will work in pairs to predict, run, investigate, and modify. The lesson will conclude with an activity requiring the learners to trace the value of a variable in an algorithm.	• Define a variable as a name that refers to data being stored by the computer • Recognise that computers follow the control flow of input/process/output • Predict the outcome of a simple sequence that includes variables • Trace the values of variables within a sequence • Make a sequence that includes a variable
Lesson 3: Selection	The focus of this lesson is to introduce learners to the concept of selection statements and how they can be used to control the flow of a program. The lesson starts with activities that allow the learners to understand expressions that evaluate to 'true' or 'false'. This will be followed by a PRIMM activity using another version of the 'Chat with Big Ed' program from the last lesson, this time using selection (**If** statements). The lesson will finish with a Parson's Problem that requires the learners to rearrange code to form a working program.	• Define a condition as an expression that will be evaluated as either true or false • Identify that selection uses conditions to control the flow of a sequence • Identify where selection statements can be used in a program • Modify a program to include selection

Lesson 4: Operators	This lesson will build on the previous lesson by introducing the use of logical and comparison operators to use in selection statements. The learners will start by following Scratch code and working out what the program will output given different inputs. They will be introduced to logical and comparison operators before taking part in an activity where they are given a playing card and have to decode if it evaluates to 'true' or 'false' using various different expressions. The learners will then build a 'Brain game' Scratch program by adding new questions to subroutines.	• Create conditions that use comparison operators (>,<,=) • Create conditions that use logic operators (and/or/not) • Identify where selection statements can be used in a program that include comparison and logical operators
Lesson 5: Count-controlled iteration	In this lesson learners will be introduced to the concept of iteration, the examples will be specifically focused on count-controlled iteration. The learners will be given an inefficient program and be asked to spot patterns and repetition. They will be taken through a live coding demonstration of taking their inefficient program and adding iteration to make it more efficient. Learners will then use pair programming to create a Scratch version of the nursery rhyme *Ten Green Bottles* using count-controlled iteration. Finally learners will be introduced to the concept of debugging and they will be given a program to debug by tracing the value of the variables.	• Define iteration as a group of instructions that are repeatedly executed • Describe the need for iteration • Identify where count-controlled iteration can be used in a program • Implement count-controlled iteration in a program • Detect and correct errors in a program (debugging)
Lesson 6: Problem-solving	This is the final lesson of the first unit of programming in Year 7. The lesson starts with a game of 'Beat the teacher' where the learners are required to write down as many as the key words relating to this unit that they can. After a minute, the teacher will also play and the learners will see if they'd written down more words than the teacher. The main activity for the lesson will be learners' main summative assessment task where they are required to independently work	• Independently design and apply programming constructs to solve a problem (subroutine, selection, count-controlled iteration, operators, and variables)

	through tasks to complete a dance move game. The plenary for the lesson allows the learners an opportunity to reflect and assess the skills that they have developed throughout the unit.	

Progression

Please see the learning graph for this unit for more information about progression.

Note: this also covers objectives for Year 7 — Programming II

Curriculum links

National curriculum links

- Use two or more programming languages, at least one of which is textual, to solve a variety of computational problems; make appropriate use of data structures (e.g. lists, tables, or arrays); design and develop modular programs that use procedures or functions
- Understand several key algorithms that reflect computational thinking; use logical reasoning to compare the utility of alternative algorithms for the same problem
- Understand simple Boolean logic (e.g. and, or, and not)
- Create, reuse, revise, and repurpose digital artefacts for a given audience, with attention to trustworthiness, design, and usability

Assessment

Summative assessment

The final lesson of the unit requires learners to complete a set of tasks using a Scratch program. Please see the assessment rubric document for this unit, and the Scratch Program Moves like Jim (ncce.io/moveJim).

Digital assessment

Student Knowledge Question Banks are available from the Teach Computing website, presented as self-marking multiple-choice questions as Microsoft / Google forms.

Multiple choice questions

This unit contains two homework activities (in Lesson 2 and Lesson 5) that ask a set of multiple choice questions. The intention of the questions is to help you pick up on which (if any) misconceptions have been picked up by the learners so that they can be addressed before moving on. Alternatively, you could ask the full set of questions at the end of the unit to help inform planning ahead of the next unit.

Subject knowledge

This unit focuses on the development of the following key techniques:

- Sequencing
- Variables
- Selection
- Operators
- Count-controlled iteration

Scratch is used throughout the unit so it is important that you are comfortable with the language. In order to get a feel for the level of skill required, why not try the activities in the unit yourself before using them with your learners?

Enhance your subject knowledge to teach this unit through the following training opportunities:

Online training courses

- Introduction to programming with Scratch
- Secondary Programming Pedagogy: Inspiring computing teaching (ncce.io/secondarypedagogy)
- Programming 101 (ncce.io/prog101)
- Programming 102 (ncce.io/progr102)

Remote courses

- Foundation knowledge of computer science for KS3 and GCSE

Misconceptions

Incorrect misconceptions can be easily established. Below are some common misconceptions identified within this unit of work. These misconceptions are provided to aid the teacher in planning this unit of work.

Common misconception:	**Guidance on how to overcome the misconception:**
1. Students may think Scratch is a low-level language because it's easy to use instead of a high-level language	1. Define abstraction and describe a high-level language like scratch as having a high level of abstraction like a bird’s eye view of how the CPU behaves through the scripts and sprites.
2. Expecting an IF block to check a condition whenever it happens instead of just immediately when the green flag is clicked	2. Practice debugging code, using paired programming to avoid common mistakes and celebrating those who demonstrate resilience

Request a Computing Ambassador

A Computing Ambassador can help to bring computing alive and provide real-life examples of computing careers, creating an invaluable link to industry. For many years, Ambassadors have been an incredible resource to schools across the UK and can be requested to support STEM skills across the curriculum. It's easy to request a Computing Ambassador (via the STEM Ambassador platform). Just create an account or sign in to your dashboard, fill in the details and publish your activity. STEM Ambassador Request

You can use the example requests, simply copy and paste the text into the portal to find the perfect Computing Ambassador to work with.

Example requests:
A day in the life of a computer programmer Year 7 (11-12) are about to embark on their first programming unit of work where they will compare how humans and computers understand instructions. We require someone with programming experience who is familiar with Scratch. You will be able to give an overview of what programming is and how it is used in your specific field with examples that can be understood by an 11-year-old. Skills and knowledge required: • Defining a sequence • Predict outcomes of a simple sequence • Modify a sequence
Computational Thinking and Decomposition in the real world As part of Year 7 (11-12) student's introduction to programming we would like someone with programming experience to demonstrate how computational thinking and decomposition is used in the development of applications to allow students to make links between their classroom-based learning and the real world. Skills and knowledge required: • Differences between how humans and computers understand instructions

- How decomposition is used to break problems down into manageable subproblems
- Purpose of subroutines
- Examples of real-world projects

Programming – The basics using Scratch

Year 7 (11-12) require someone with experience of using Scratch to support the introduction of key programming themes.

Skills and knowledge required:

- Assigning variables
- Operators
- Sequencing
- Selection
- Count-controlled iteration.

The aim of this unit of work is for students to independently design and apply the programming constructs they have learned to solve a particular problem.

Resources are updated regularly — the latest version is available at: ncce.io/tcc.

Attribution statement

This resource was created by Raspberry Pi Foundation and updated by STEM Learning for the National Centre for Computing Education.

The contents of this resource are available for use under the Open Government License (OGL v3) meaning you can copy, adapt, distribute and publish the information. You must acknowledge the source of the Information in your product or application, by attributing Raspberry Pi Foundation and STEM Learning as stated here and are asked to provide a link to the OGL v3.

The original version can be made available on request via info@teachcomputing.org.